



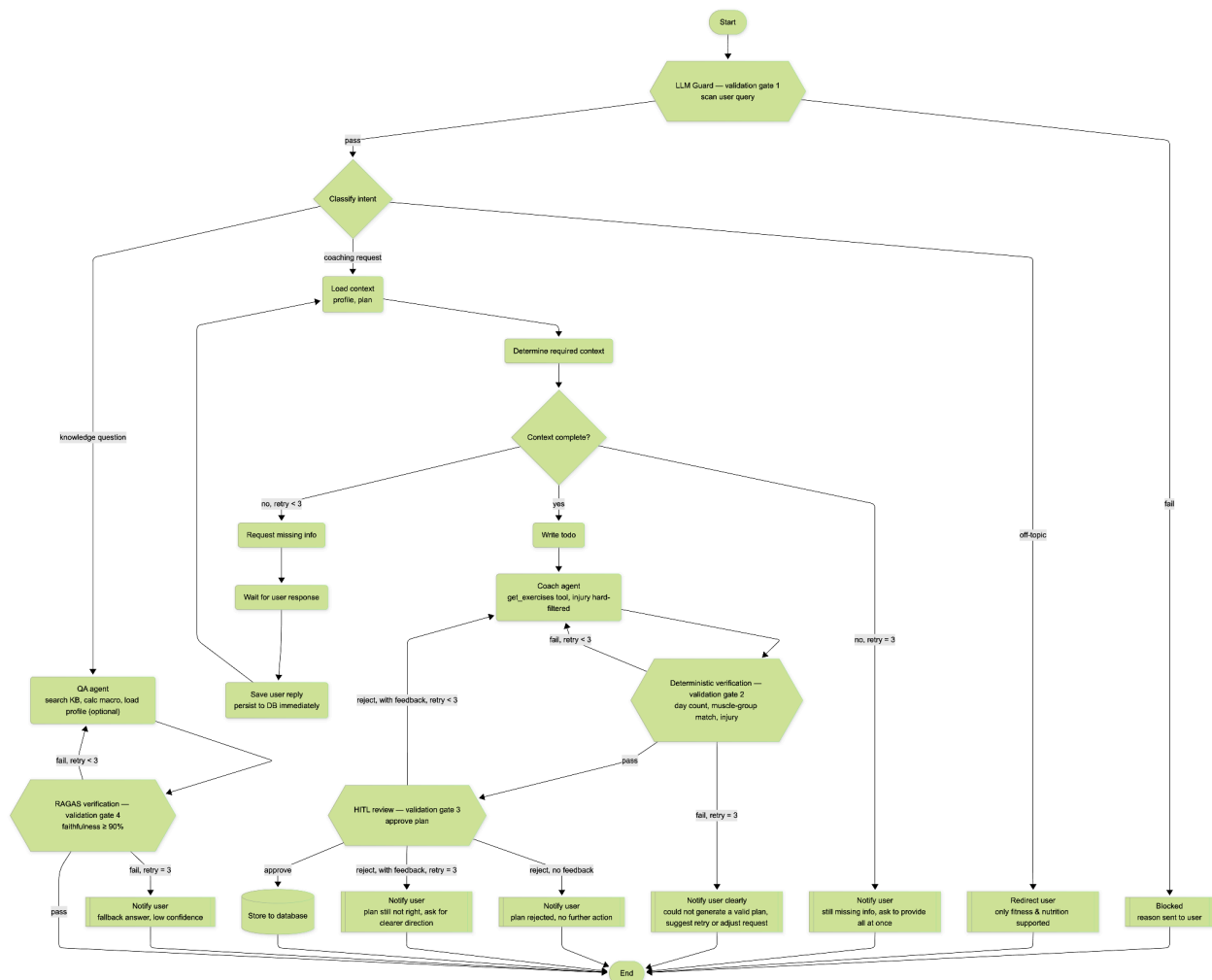
www.agilityio.com

Implementation Detail Document

Aug 18, 2026

Graph Design

[Link](#)



Node Design

Node	Purpose
llm_guard	Scan input
blocked	Return message for user and stop immediately
classify_intent	Classify user into three main intent: coaching, qa and off_topic
off_topic_response	Return reject message for off topic
load_context	Load profile and plan of user
determine_context	Identify missing field to create plan
request_missing_info	Ask user to provide missing fields
wait_for_user	interrupt() - pause graph and wait for user provide data
save_user_data	Save user info after
write_todo	Write a todo list for coach agent
coach_agent	Agent for create plain
deterministic_verification	Check schema/macro/volume/safety by fixed rule no use LLM
notify_fail	Notify user when verification fail more than 3 time
hitl_review	interrupt() wait for approve/reject
hitl_rejected_no_feedback	Handle reject with no feedback case, stop immediately

hitl_exhausted	Handle reject with feedback case but attempted over 3 times, send message inform and top
qa_agent	Agent for answer knowledge question
ragas_verification	Validation faithfulness => 90%
qa_fallback	Fallback when validation faithfulness fail 3 times, response to user the data is untrusted

Edge / Routing Design

From	Condition	To
llm_guard	blocked	blocked
llm_guard	pass	classify_intent
classify_intent	qa	qa_agent
classify_intent	coaching	load_context
classify_intent	off_topic	off_topic_response
load_context	context complete	write_todo
load_context	missing fields	determine_context
determine_context	missing fields exist	request_missing_info
save_user_data	data updated	load_context
write_todo		coach_agent
coach_agent	success	deterministic_verification
deterministic_verification	pass	hitl_review
deterministic_verification	fail & retry < 3	coach_agent
deterministic_verification	fail & retry >= 3	notify_fail
hitl_review	approve	end
hitl_review	reject + feedback	coach_agent

hitl_review	reject + no feedback	hitl_rejected_no_feedback
hitl_review	reject + retry >= 3	hitl_exhausted
qa_agent	answer generated	ragas_verification
ragas_verification	>= 0.9 faithfulness	end
ragas_verification	< 0.9 & retry < 3	qa_agent
ragas_verification	< 0.9 & retry >= 3	qa_fallback

State Design

```
class GraphState(AgentState):
    # =====
    # Input
    # =====
    messages: Required[Annotated[list[AnyMessage], add_messages]]
    user_query: str
    user_id: str

    # =====
    # Intent & Guard
    # =====
    intent: Optional[Literal["coaching", "qa", "off_topic"]]
    guard_blocked: bool
    block_reason: Optional[str]

    # =====
    # User Context
    # =====
    profile: Optional[dict]
    plan: Optional[dict]
    context_complete: bool
    missing_fields: list[str]

    # =====
```

```

# Coaching
# =====
todo: Optional[list[dict]]
coach_retry_count: int
verification_result: Optional[dict]

# =====
# HITL
# =====
hitl_decision: Optional[Literal["approve", "reject"]]
hitl_feedback: Optional[str]
hitl_retry_count: int

# =====
# QA / RAG
# =====
qa_answer: Optional[str]
retrieved_context: Optional[list[dict]]
ragas_score: Optional[float]
qa_retry_count: int

# =====
# Final Output
# =====
final_message: Optional[str]

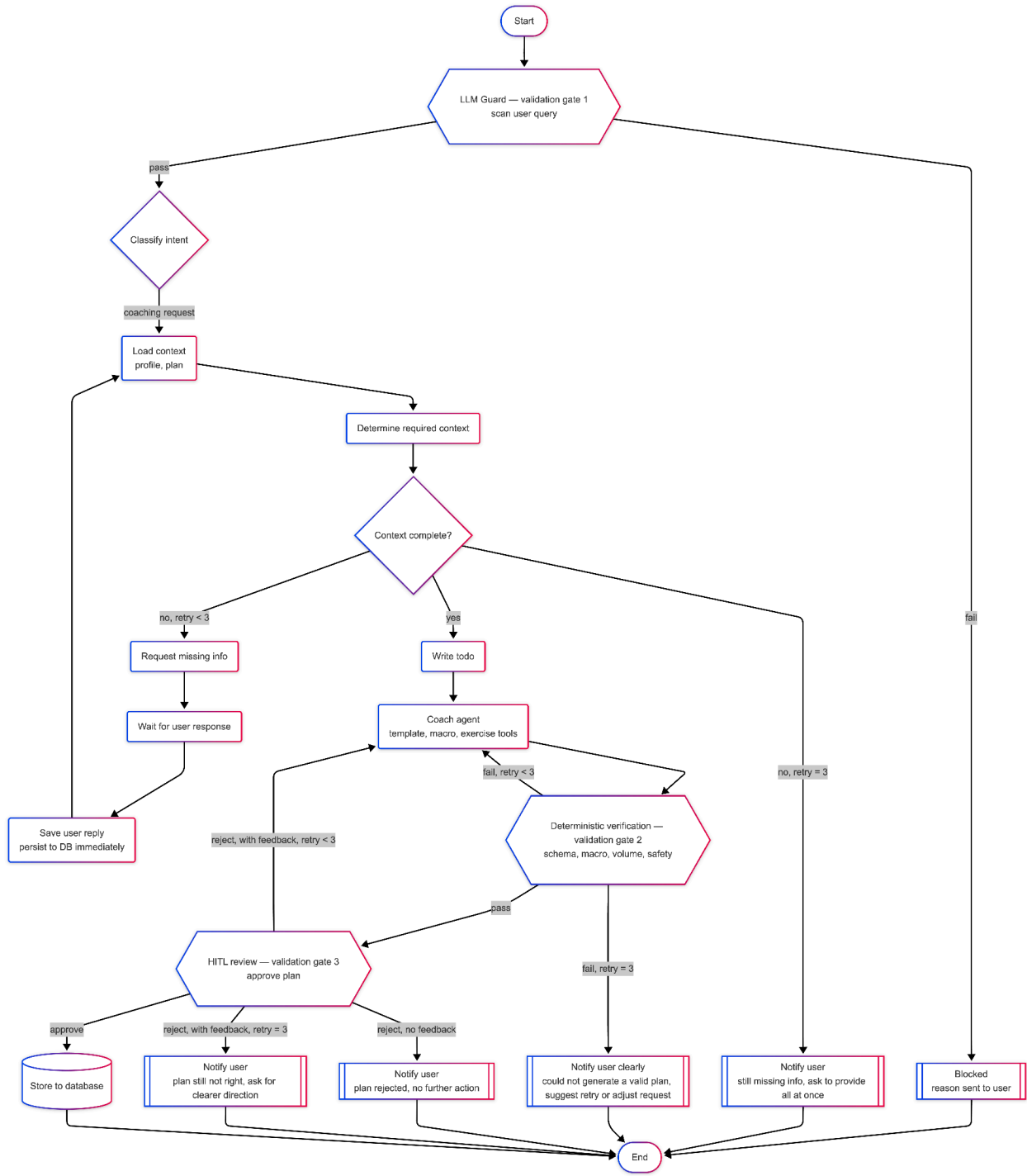
```

Agent Design

Coach Agent

Generate or modify a personalized training plan based on the user's profile, goal, constraints, and todo list.

Workflow



Input:

- user_query
- profile
- plan (if an existing plan is available)
- todo
- messages
-

Output:

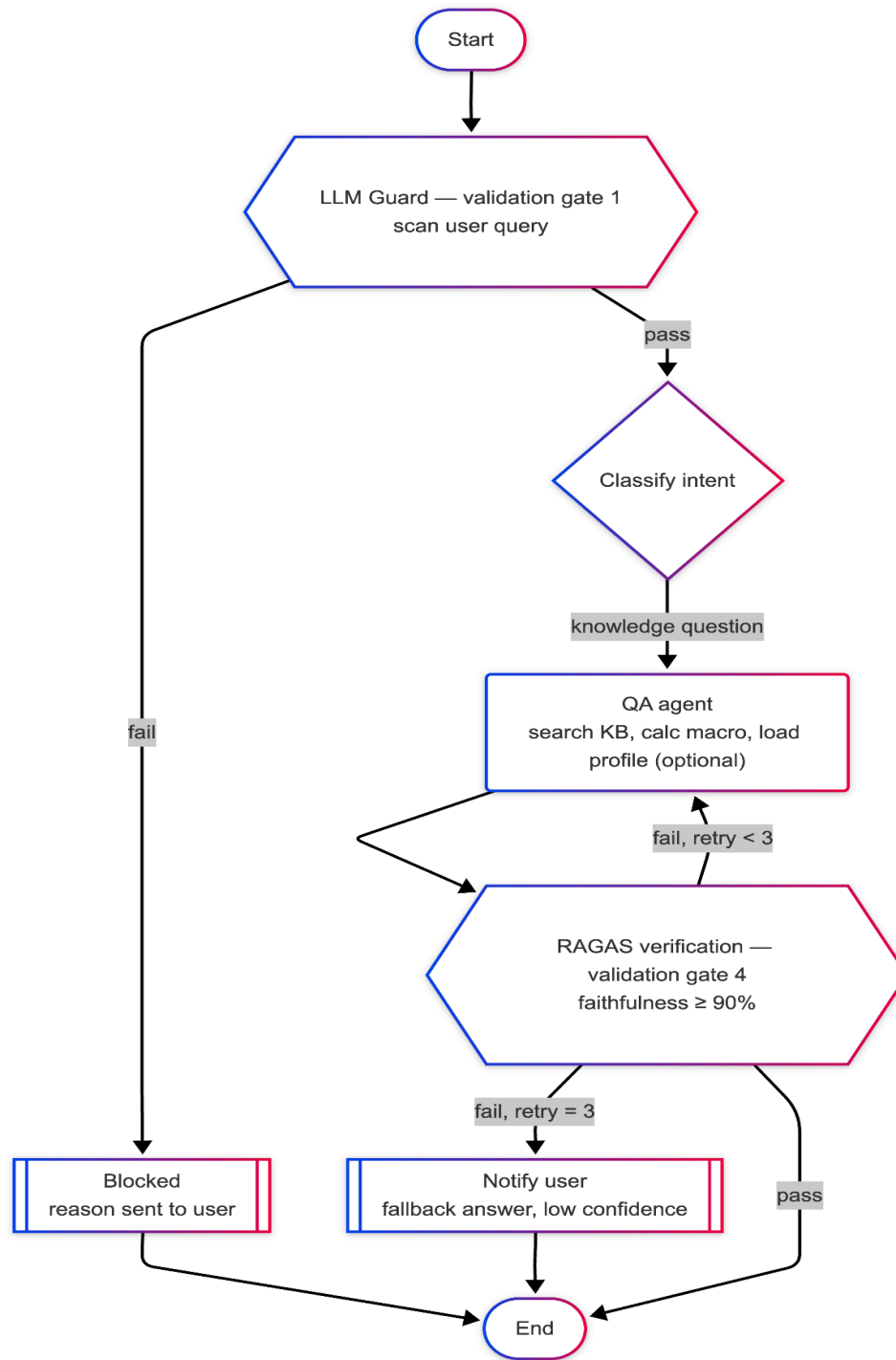
- Training plan
 - Goal
 - Calories
 - Macro
 - Training days
 - Exercise

Tool	Purpose
load_template	Retrieve a suitable training plan template
load_exercise	Retrieve exercises matching training requirements and constraints
calc_macro	Calculate calorie and macro targets

QA Agent

Answer nutrition and injury-related knowledge questions using the retrieved local knowledge base.

Workflow



Input:

- user_query
- profile (when relevant)
- messages

Output:

- answer

Tool	Purpose
search_knowledge	Retrieve relevant passages from the local knowledge base
load_profile	Load user profile when the answer requires user-specific context
calc_macro	Calculate calorie and macro targets to answer question relative to user

Verification Design

Deterministic Verification

Validate the training plan using fixed rules without LLM.

Checks:

- Schema completeness
- Macro consistency
- Training volume
- Training schedule
- Exercise availability
- User constraints
- Safety constraints

Threshold:

- If verification fails and `coach_retry_count < 3`, return the validation errors to `coach_agent` for revision.
- If verification fails after 3 attempts, route to `notify_fail`.

RAGAS Verification

Validate QA answer faithfulness against retrieved context.

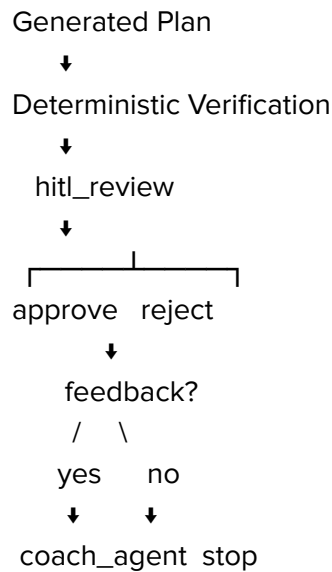
Threshold:

- `ragas_score >= 0.9` → Pass
- `ragas_score < 0.9` → Retry QA Agent

If the score remains below 0.9 after 3 attempts, route to `qa_fallback`.

HITL Design

Review Flow



Approve

Return the approved plan to the user.

Reject with Feedback

Pass `hitl_feedback` to `coach_agent` and regenerate/revise the plan.

Increment `hitl_retry_count`.

Reject without Feedback

Route to `hitl_rejected_no_feedback` and stop the workflow.

Retry Limit

If the user rejects the plan with feedback more than 3 times, route to `hitl_exhausted` and stop the workflow.

Interrupt

`hitl_review` uses `interrupt()` to pause the graph and wait for the user's decision.

The checkpoint preserves the current `GraphState` so the graph can resume after the user responds.

Memory & Persistence Design

Shortterm Memory

Checkpointers:

- `AsyncPostgresSaver`
- Store graph checkpoints in PostgreSQL.
- Preserve `GraphState` during graph execution.
- Support graph resume after `interrupt()`.

Short-term memory is used for **current workflow execution**, not persistent user knowledge.

Longterm Memory

Pgvector

- `knowledge_chunks` stores embedded knowledge from nutrition and injury documents.

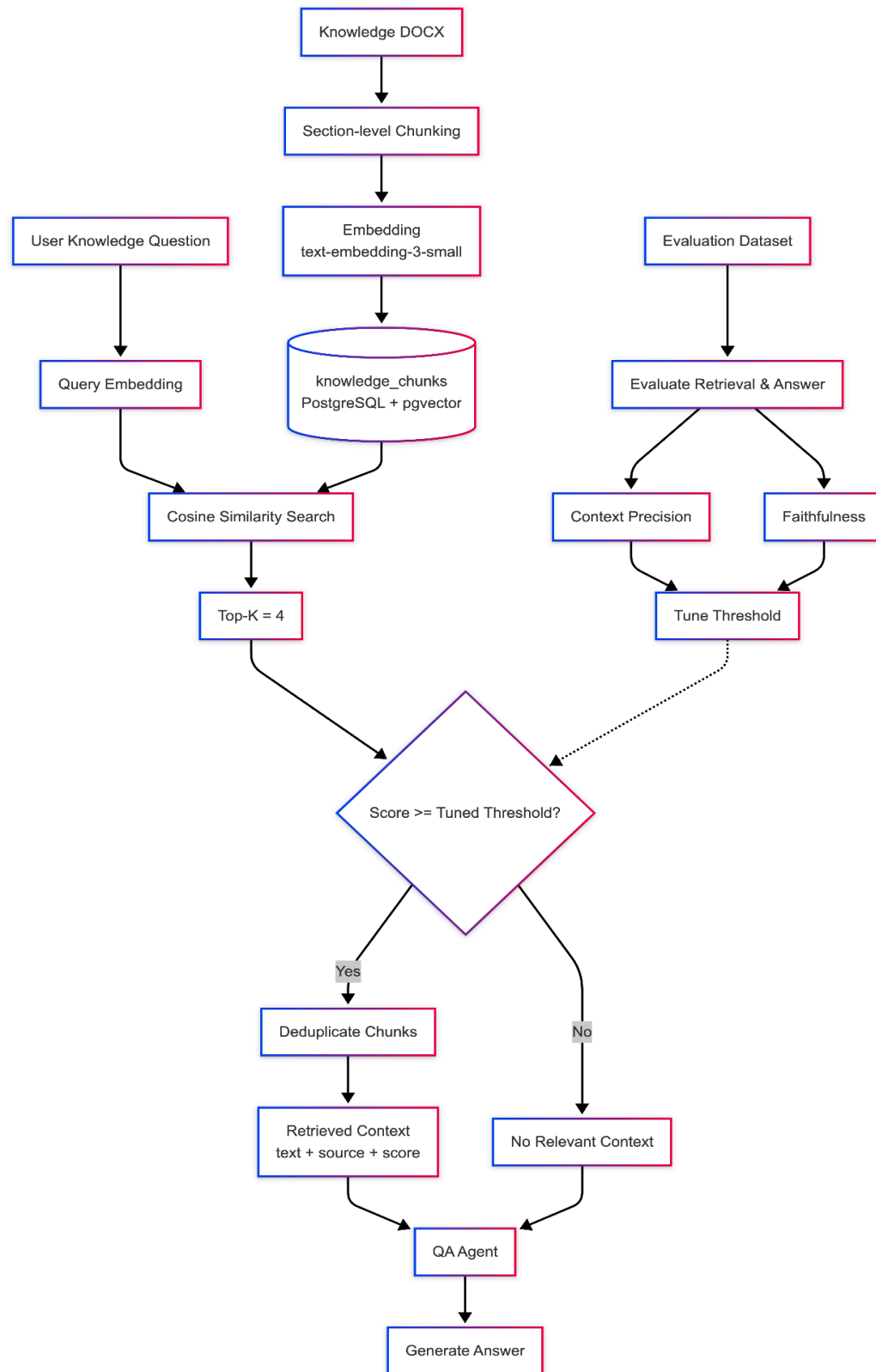
PostgresStore

Store structured user-specific information:

- **User preferences:** preferred workout schedule, favorite exercises, dietary preferences, preferred response style.
- **Accumulated knowledge:** behavioral patterns learned over time, such as tendency to skip workouts longer than 60 minutes or better adherence to 4-day plans.
- **Facts:** user information such as name, age, weight, sex, activity level, goal, equipment, and injury.

RAG Design

Workflow



Retrieval

- Embedding model: `text-embedding-3-small`
- Similarity: Cosine similarity
- `top_k = 4`
- Filter using a tuned similarity threshold
- Remove duplicate or highly overlapping chunks

Retrieval Output

```
{  
  "text": str,  
  "source": str,  
  "score": float  
}
```

Return `[]` when no passage meets the configured threshold.

No Relevant Context

If no relevant passage is retrieved:

- QA Agent must not answer using unsupported knowledge.
- Return a fallback response indicating that sufficient trusted information is unavailable.

Evaluation

Offline evaluation:

- Context Precision
- Faithfulness

Use evaluation results to tune the retrieval threshold and improve retrieval quality.

Online validation:

- RAGAS faithfulness ≥ 0.9 is required before returning the QA answer.

Error Handling

Error	Handling
LLM failure	Retry according to agent retry policy

Tool failure	Retry agent/tool call
Invalid agent output	Retry agent with validation error
Guard failure	Block request
Deterministic verification failure	Retry Coach Agent up to 3 times
RAGAS failure	Retry QA Agent up to 3 times
HITL rejection with feedback	Revise plan
HITL rejection without feedback	Stop workflow
HITL retry exhausted	Stop workflow and notify user
No relevant RAG context	Return untrusted/insufficient-context response

All retry limits are controlled by the corresponding retry counter in [GraphState](#).

Observability

Observability

Use **Langfuse** to trace each graph execution and major workflow step.

Track:

- [run_id](#)
- [user_id](#)
- [intent](#)
- Node execution
- Agent execution
- Tool calls
- Retrieved chunks and similarity scores
- Retry counts
- Verification results
- RAGAS score
- Latency
- Final result

Pydantic Data Models and Specification

Enumerations (ENUMS)

Sex

Description: User biological sex

Value	Description
MALE	Male
FEMALE	Female

ActivityLevel

Description: Daily physical activity level

Value	Description
SEDENTARY	Sedentary (desk job, little to no exercise)
LIGHT	Lightly active (light exercise/sports 1-3 days/week)
MODERATE	Moderately active (moderate exercise/sports 3-5 days/week)
VERY_ACTIVE	Very active (hard exercise/sports 6-7 days a week)
EXTRA_ACTIVE	Extra active (very hard exercise, physical job or training 2x/day)

FitnessGoal

Description: User fitness goal

Value	Description
FAT_LOSS	Fat loss
MUSCLE_GAIN	Muscle gain / Hypertrophy
MAINTENANCE	Weight/Fitness maintenance
STRENGTH	Strength building
GENERAL_FITNESS	General fitness and health improvement

BodyRegion

Description: Target body region

Value	Description
UPPER	Upper body
LOWER	Lower body
FULL_BODY	Full body
CORE	Core region

MuscleGroup

Description: Specific muscle groups

Value	Description
CHEST	Chest

BACK	Back
SHOULDERS	Shoulders
BICEPS	Biceps
TRICEPS	Triceps
FOREARMS	Forearms
QUADS	Quadriceps
HAMSTRINGS	Hamstrings
GLUTES	Glutes
CALVES	Calves
ABS	Abdominals
LOWER_BACK	Lower back
HIP	Hip muscles

MovementPattern

Description: Exercise movement patterns

Value	Description
HORIZONTAL_PUSH	Horizontal Push (e.g., Bench Press, Push-up)
HORIZONTAL_PULL	Horizontal Pull (e.g., Barbell Row, Cable Row)
VERTICAL_PUSH	Vertical Push (e.g., Overhead Press)
VERTICAL_PULL	Vertical Pull (e.g., Pull-up, Lat Pulldown)

SQUAT	Squat / Knee Dominant (e.g., Back Squat)
HINGE	Hinge / Hip Dominant (e.g., Deadlift, Hip Thrust)
LUNGE	Lunge / Single Leg (e.g., Walking Lunge)
CARRY	Carry (e.g., Farmer's Carry)
ISOLATION	Isolation / Single-joint
ROTATION	Rotational movement
ANTI_ROTATION	Anti-rotation (e.g., Pallof Press)
FLEXION	Flexion
EXTENSION	Extension

EquipmentType

Description: Available equipment types

Value	Description
BODYWEIGHT	Bodyweight
BARBELL	Barbell
DUMBBELL	Dumbbell
CABLE	Cable machine
MACHINE	Weight machine
KETTLEBELL	Kettlebell
RESISTANCE_BAND	Resistance band

SMITH_MACHINE	Smith machine
BENCH	Exercise bench
OTHER	Other equipment

DifficultyLevel

Description: Exercise difficulty level

Value	Description
BEGINNER	Beginner level
INTERMEDIATE	Intermediate level
ADVANCED	Advanced level

InjuryStatus

Description: Exercise difficulty level

Value	Description
ACTIVE	Active injury / currently causing pain
RECOVERING	Recovering / in rehabilitation
RESOLVED	Fully resolved / healed

RestrictionAction

Description: Restriction level for movements/exercises

Value	Description
-------	-------------

ACTIVE	Active injury / currently causing pain
RECOVERING	Recovering / in rehabilitation
RESOLVED	Fully resolved / healed

Data Models (Pydantic Schemas)

MovementRestriction

Description: Defines a movement or exercise pattern that should be avoided or limited because of an injury.

Field	Type	Description	Required
movement_pattern	MovementPattern	Affected movement pattern	Yes
action	RestrictionAction	Restriction action level (PROHIBITED, LIMITED, ALLOWED)	Yes
reason	str	Reason for the restriction	No (Optional)

Injury

Description: User's current or previous injury information.

Field	Type	Description	Required
body_part	str	Affected body part, e.g. shoulder, knee, lower_back	Yes
status	InjuryStatus	Injury status	Yes (Default: ACTIVE)
severity	str	Injury severity description	No (Optional)
restrictions	List[MovementRestriction]	Movement restrictions caused by the injury	Yes (Default: [])

notes	str	Additional notes on injury	No (Optional)
-------	-----	----------------------------	---------------

UserEquipment

Description: Equipment available to the user.

Field	Type	Description	Required
equipment	List[EquipmentType]	List of available standard equipment	Yes (Default: [])
other_equipment	List[str]	List of additional/custom equipment	Yes (Default: [])

UserProfile

Description: User information used by the Coach Agent when generating a workout plan.

Field	Type	Description	Required
age	int	User age in years (valid range: 13 to 100)	Yes
sex	Sex	Biological sex (male, female, other)	Yes
height_cm	float	User height in centimeters (> 0)	Yes
current_weight_kg	float	Current body weight in kilograms (> 0)	Yes
target_weight_kg	float	Target body weight in kilograms (> 0)	No (Optional)
activity_level	ActivityLevel	Daily activity level	Yes
goal	FitnessGoal	Primary fitness goal	Yes

training_days_per_week	int	Number of training days per week (1 - 7)	Yes
injuries	List[Injury]	Current or relevant previous injuries	Yes (Default: [])
available_equipment	UserEquipment	Equipment available to the user	Yes (Default: UserEquipment)
preferences	List[str]	Optional user preferences, e.g. prefers dumbbells	Yes (Default: [])
notes	str	Additional user notes	No (Optional)

ExerciseContraindication

Description: Describes an injury/body-part condition under which an exercise should not be selected.

Field	Type	Description	Required
body_part	str	Affected body part	Yes
movement_patterns	List[MovementPattern]	Contraindicated movement patterns	Yes (Default: [])
reason	str	Reason for contraindication	No (Optional)

ExerciseMuscleTarget

Description: Muscle involvement of an exercise.

Field	Type	Description	Required
muscle	MuscleGroup	Target muscle group	Yes
priority	str	Muscle involvement role ('primary' or 'secondary')	Yes

Exercise

Description: Structured exercise definition available to the Coach Agent.

Field	Type	Description	Required
id	str	Unique exercise identifier	Yes
name	str	Exercise name	Yes
description	str	Detailed exercise description	No (Optional)
body_region	BodyRegion	Primary body region targeted	Yes
primary_muscles	List[MuscleGroup]	List of primary target muscles (at least 1 required)	Yes (Min items: 1)
secondary_muscles	List[MuscleGroup]	List of secondary target muscles	Yes (Default: [])
movement_pattern	MovementPattern	Primary movement pattern	Yes
equipment	List[EquipmentType]	Required equipment list	Yes (Default: [])
difficulty	DifficultyLevel	Exercise difficulty level	Yes
contraindications	List[ExerciseContraindication]	List of injury contraindications	Yes (Default: [])
instructions	List[str]	Step-by-step instructions	No (Optional)
notes	str	Additional notes	No (Optional)

ExerciseSlot

Description: A slot in the workout template that the Coach Agent needs to fill with an exercise.

Field	Type	Description	Required
slot_id	str	Identifier for the exercise slot	Yes
exercise_type	str	Desired exercise type description	No (Optional)
target_muscles	List[MuscleGroup]	Target muscle groups for this slot	Yes (Default: [])
allowed_movement_patterns	List[MovementPattern]	Allowed movement patterns	Yes (Default: [])
excluded_movement_patterns	List[MovementPattern]	Excluded movement patterns	Yes (Default: [])
required_body_region	BodyRegion	Required body region constraint	No (Optional)
alternatives_allowed	bool	Whether alternative exercises are allowed	Yes (Default: True)
notes	str	Slot specific notes	No (Optional)

WorkoutDayTemplate

Description: Defines the structure and constraints of one training day.

Field	Type	Description	Required
day_number	int	Day sequence number (>= 1)	Yes
name	str	Workout day name (e.g. Leg Day, Upper Push)	Yes

body_region	BodyRegion	Primary body region for the day	Yes
target_muscles	List[MuscleGroup]	Target muscle groups for the day	Yes (Default: [])
exercise_slots	List[ExerciseSlot]	List of exercise slots (at least 1 required)	Yes (Min items: 1)
notes	str	Notes for the training day	No (Optional)

WorkoutTemplate

Description: Overall workout plan template defining structure for the Coach Agent to select actual exercises.

Field	Type	Description	Required
id	str	Unique template identifier	Yes
name	str	Template name	Yes
description	str	General template description	No (Optional)
training_days	List[WorkoutDayTemplate]	List of training days (at least 1 required)	Yes (Min items: 1)
notes	str	Overall template notes	No (Optional)

Estimation

Date	Task	Estimate time (hour)	Actual time (hour)	Status
Minestone 1 — Project Initialization & Setup (4h)				
21/08	Init LangGraph project structure (repo layout, package/dependency setup)	1		Todo ▾
	Setup PostgreSQL instance + AsyncPostgresSaver checkpointer config	1		Todo ▾
	Enable pgvector extension & base PostgresStore setup	0.5		Todo ▾
	Define GraphState schema (input, guard, context, coaching, HITL, QA, output)	0.5		Todo ▾
	Setup Docker + Docker Compose (app, Postgres, pgvector)	0.5		Todo ▾
	Base Langfuse project/tracing wiring	0.5		Todo ▾
Minestone 2 — Guard & Intent Classification (4h)				
21/08	Implement llm_guard node (integrate llm-guard input scanning)	1		Todo ▾
	Implement blocked node (return block message, stop immediately)	0.5		Todo ▾
	Implement classify_intent node (coaching / qa / off_topic)	0.5		Todo ▾
	Implement off_topic_response node	0.5		Todo ▾
	Wire routing edges: llm_guard → classify_intent → 3 branches	0.5		Todo ▾
	Unit tests: guard scanning + intent classification accuracy	1		Todo ▾
Minestone 3 — Context Loading Branch (5h)				
22/08	Implement load_context node (load profile & plan)	1		Todo ▾

	Implement determine_context node (identify missing fields)	1		Todo ▾
	Implement request_missing_info node	0.5		Todo ▾
	Implement wait_for_user node (interrupt() pause/resume)	1		Todo ▾
	Implement save_user_data node (persist reply to DB before reload)	1		Todo ▾
	Wire routing edges for context-complete / missing-fields branch	0.5		Todo ▾
Minestone 4 — Coach Agent & Tools (5h)				
22/08	Implement write_todo node	1		Todo ▾
	Implement coach_agent core (LangGraph agent + tool binding)	1		Todo ▾
	Implement load_template tool	0.5		Todo ▾
	Implement load_exercise tool	0,5		Todo ▾
24/08	Implement calc_macro tool (shared calorie/macro calculation)	0.5		Todo ▾
	Implement Pydantic schemas (UserProfile, Injury, Exercise, WorkoutTemplate, etc.)	0.5		Todo ▾
	Implement enums (Sex, ActivityLevel, FitnessGoal, MovementPattern, Equipment, etc.)	0.5		Todo ▾
	Unit tests for coach agent + tools	0.5		Todo ▾
Minestone 5 — Deterministic Verification Gate (7h)				
24/08	Implement schema completeness check	1		Todo ▾
	Implement macro consistency check	1		Todo ▾
	Implement training volume & schedule check	1		Todo ▾
	Implement exercise availability & user constraints check	1		Todo ▾

	Implement safety constraints check	1		Todo ▾
	Implement coach_retry_count logic & pass/fail routing	1		Todo ▾
25/08	Implement notify_fail node (retry >= 3)	0.5		Todo ▾
	Unit tests for deterministic verification rules	0.5		Todo ▾
Minestone 6 — HITL Review Gate (5h)				
25/08	Implement hitl_review node (interrupt() approve/reject)	1		Todo ▾
	Implement approve routing → end	0.5		Todo ▾
	Implement reject + feedback routing → coach_agent revision	1.5		Todo ▾
	Implement hitl_rejected_no_feedback node (stop immediately)	0.5		Todo ▾
	Implement hitl_exhausted node (reject + retry >= 3)	0.5		Todo ▾
	Implement hitl_retry_count tracking	0.5		Todo ▾
	Integration test: full HITL approve/reject/exhausted loop	0.5		Todo ▾
Minestone 7 — QA Agent & RAG Pipeline (8h)				
25/08	Implement qa_agent node (LangGraph agent) + tools binding	1		Todo ▾
	Implement search_knowledge tool (pgvector retrieval, top_k=4, cosine similarity)	1		Todo ▾
26/08	Implement load_profile tool for QA context	0.5		Todo ▾
	Build embedding pipeline (text-embedding-3-small) for knowledge_chunks	1		Todo ▾
	Implement similarity threshold filtering & duplicate/overlap removal	1		Todo ▾
	Implement ragas_verification node (faithfulness scoring)	1		Todo ▾

	Implement qa_fallback node (untrusted/insufficient-context response)	1		Todo ▾
	Wire routing: score >= 0.9 pass / < 0.9 retry / < 0.9 & retry >= 3 fallback	0.5		Todo ▾
	Unit tests for QA agent + RAG retrieval pipeline	1		Todo ▾
Minestone 8 — Memory & Persistence (6h)				
26/08	Integrate AsyncPostgresSaver checkpointer + resume-after-interrupt tests	1		Todo ▾
27/08	Setup PostgresStore for preferences, accumulated knowledge, facts	2		Todo ▾
	Create knowledge_chunks table + pgvector index	1		Todo ▾
	Data migration scripts / seed nutrition & injury knowledge base	1		Todo ▾
	Integration test: short-term (checkpoint) vs long-term memory separation	1		Todo ▾
Minestone 9 — Observability & Error Handling (4h)				
27/08	Integrate Langfuse tracing across nodes, agents, and tool calls	1		Todo ▾
	Track run_id, user_id, intent, retry counts, verification & RAGAS results	1		Todo ▾
	Implement centralized error handling (LLM/tool failure retry policy)	1		Todo ▾
28/08	Latency tracking & final result logging	1		Todo ▾
Minestone 10 — Integration Testing, Refactor & Bug Fixing (8h)				
28/08	End-to-end test: full coaching flow (intent → context → plan → verify → HITL)	3		Todo ▾
	End-to-end test: full QA flow (intent → retrieval → answer → RAGAS)	3		Todo ▾
	Code review & cleanup	2		Todo ▾
Total Estimate		56		
Total Day		7		

